

Predicting Project Velocity in XP Using a Learning Dynamic Bayesian Network Model

Peter Hearty, Norman Fenton, David Marquez, and Martin Neil

Abstract—Bayesian networks, which can combine sparse data, prior assumptions, and expert judgment into a single causal model, have already been used to build software effort prediction models. We present such a model of an Extreme Programming environment and show how it can learn from project data in order to make quantitative effort predictions and risk assessments without requiring any additional metrics collection program. The model's predictions are validated against a real-world industrial project, with which they are in good agreement.

Index Terms—Bayesian nets, causal models, extreme programming, risk assessment.

1 INTRODUCTION

EXTREME Programming (XP) is one of several iterative approaches to software development, collectively known as “Agile” methods [32]. It consists of a collection of values, principles, and practices as outlined by Beck et al. [21], [22], [23]. These include most notably: iterative development, pair programming, collective code ownership, frequent integration, onsite customer input, unit testing, and refactoring.

XP emphasizes a lightweight, often informal approach. There are no large-scale requirements, analysis, and design phases, and so there are none of the traditional metrics associated with the requirements or design phases, such as function points [37]. Instead, the customer and development team agree a series of user stories (described later) that concisely define the requirements. User story sizes can be estimated using relative size measures such as story points that can be used to create initial project plans [38]. However, the definition of a user story is not as well defined as a function point. As such, user stories are limited in their ability to predict effort or quality. Yet, managers of XP projects have just as great a need for such predictions as managers on any other software project.

Effort estimates for user stories only cover time spent working directly on the user stories. They do not cover other overheads. Managers need to know how accurate developers' estimates are and how they translate to calendar time. Only then can a project manager determine how long the project will actually take to complete. This paper addresses this problem by exploiting ideas on causal modeling that have led to improved effort and quality prediction for traditional software development.

Fenton and Neil [11] explained the rationale behind creating causal models of the software development process using Bayesian Nets (BNs). BNs offer the advantage of being able to reason in the presence of uncertainty, prior assumptions, and incomplete data. They can intermix expert judgment, statistical distributions, and observations in a single model. Further, they are able to learn from evidence in order to update their prior beliefs. In an iterative development environment, such as XP, we can take advantage of this learning capability, where information obtained from early iterations in the project can be used to adapt a model to the local environment.

This paper presents a BN model of Project Velocity (PV), the one management metric that is always available in XP. Roughly speaking, PV can be thought of as a “productive effort per iteration.” The exact definition of PV is given in Section 3.2.

We set the following key requirements that the model must satisfy:

1. It must monitor and predict PV, taking into account the impact of relevant process factors.
2. For computability reasons, the core model must be very small. This enables it to be replicated multiple times in order to represent the multiple iterations of an agile development environment.
3. The model must be able to handle different types of data for different environments. In particular, the model must handle key XP practices, while being dependent on none of them.
4. The model must be capable of replicating empirical behavior. In particular, many projects report low initial productivity, gradually rising on subsequent iterations [7], [8], [5].
5. The model must learn from data, either as a result of observations or as a result of expert judgment entered as evidence.
6. It must give useful and clear advice to managers.

The main research contribution of this paper is to introduce and validate dynamic Bayesian nets as a means of modeling iterative software development. PV data are collected from

• The authors are with Queen Mary University of London-Computer Science, Mile End Road, London E1 4NS, UK.
E-mail: {hearty, fenton, marquezd, martin}@dcs.qmul.ac.uk.

Manuscript received 15 Mar. 2008; revised 21 June 2008; accepted 29 July 2008; published online 8 Sept. 2008.

Recommended for acceptance by R. Taylor.

For information on obtaining reprints of this article, please send e-mail to: tse@computer.org, and reference IEEECS Log Number TSE-2008-03-0102.
Digital Object Identifier no. 10.1109/TSE.2008.76.

the first iteration in any XP project. This is incorporated into the model, enabling it to learn key parameters and increase the confidence of its predictions in subsequent iterations. We show that, with very little data, it is possible to correct the model's prior assumptions and quickly produce accurate models of PV with associated risk assessments. Crucially, the model does not depend upon any previous metrics collection program. The only inputs consist of data that would normally be available in any XP project.

The remainder of this paper is organized as follows: In Section 2, we discuss related models, both models of XP and other BN models of the software engineering process. Section 3 covers some of the definitions needed by the model. Section 4 presents the model itself. Section 5 covers model behavior using hypothetical data, while Section 6 validates the model using a real XP project. Section 7 discusses the implications of the model, including threats to its validity and some conclusions.

2 RELATED WORK

We discuss related work under two headings. We first examine other predictive models of XP development, before going on to describe BN models of the software development process.

2.1 Extreme Programming Predictive Models

Williams and Erdogmus [19] developed a Net Present Value (NPV) [20] model of Pair Programming (PP)—one of the key practices advocated by XP. NPV models take into account the fact that earnings in the future are worth less than the same dollar earnings today. The model combines

1. productivity rates,
2. code production rates (derived from the literature),
3. defect insertion rates, and
4. defect removal rates.

Using empirical values for PP productivity and delivered defect rates [24], [25], [26], the model predicts that PP is a “viable alternative to individual programming.”

Padberg and Müller [27] also created an NPV model of XP. Their model uses market pressure as the principle means of discounting the NPV. The model was tested under various different assumptions about performance and defect rate improvements under PP. The results indicate that the value of both of these parameters is crucial. When market pressure is high and there is sufficient improvement in both LOC and defect rates, then PP can indeed deliver an advantage.

Several groups have constructed System Dynamics (SD) models of XP. SD models a system as a collection of stocks, flows, and feedback loops and was first applied to software engineering by Abdel-Hamid [29]. Misic et al. [28] attempted to model the interaction of various XP practices. They particularly concentrated on pair programming, refactoring, test-driven development, and iterative development. Simulation results indicated that XP has an advantage when pairs worked well together and did not swap frequently.

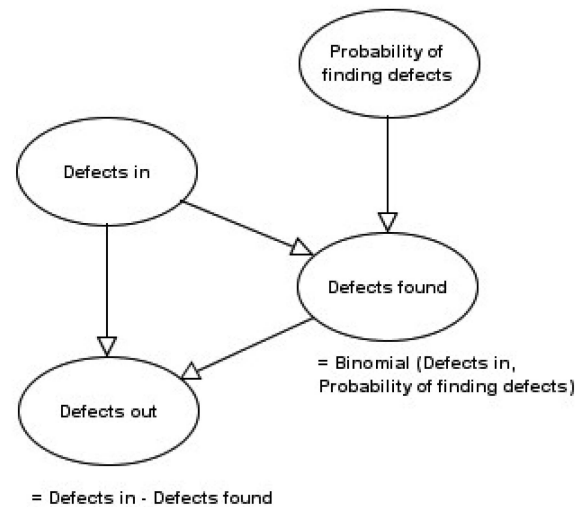


Fig. 1. An example of a Bayesian Network.

Kuppuswami et al. [30] also created an SD model. They were able to successfully simulate the flattened cost of change curve claimed by Beck [22, p. 23].

Cau et al. [33] developed a custom simulation to model the XP process, calibrated using data from a real XP project. Once calibrated, their model was able to reproduce empirically derived results [34] about the effects of test-driven development (one of the recommended XP practices).

All of the above provide explanation, insight, or validation of the XP techniques. What none claims to do is offer combined prediction and risk assessment for project managers. This can be achieved using causal models, which are now being used effectively in traditional software engineering.

2.2 Bayesian Net Models in Software Engineering

A Bayesian Net (BN) [1] is a directed acyclic graph (such as the example shown in Fig. 1), where the nodes represent random variables and the directed arcs define causal influences or functional relationships. Nodes without parents (such as the “Probability of finding defects” and “Defects in” nodes in Fig. 1) are defined through their prior probability distributions. Nodes with parents are defined through Conditional Probability Distributions (CPDs). For some nodes, the CPDs are defined through deterministic functions of their parents (such as the “Defects out” node in Fig. 1), others (such as the “Defects found” node in Fig. 1) are defined as standard probability distribution functions. Conditional independence (CI) relationships are implicit in the directed acyclic graph: All nodes are conditionally independent of their nondescendants given their parents. This general rule makes it unnecessary to list CI relationships explicitly.

As explained by Fenton and Neil [11], BN models have several advantages over regression-based models. BNs do not rely on point values of parameters that have been derived through some “best fit” procedure. Instead, the whole distribution of a variable is included. Similarly, BN models do not just predict a single value for a variable; they predict its probability distribution. By taking the marginal

distributions of variables of interest, we get a ready-made means of providing quantitative risk assessment.

When a variable is actually observed, this observation can be entered into the model. An observation reduces the marginal probability distribution for the observed variable to a probability of 1 for the observed state (or a small interval containing the value in the continuous case—in practice, the discretization chosen must be a compromise between the accuracy of the evidence added and computational feasibility) and zero otherwise. The presence of an observation updates the CPD of its children and, through Bayes theorem, the distributions of its parents. In this way, observations are propagated recursively through the model. BN models can therefore update their beliefs about probable causes and so learn from the evidence entered into the model. More information on BNs and suitable propagation algorithms can be found in [1] and [6].

Neil et al. have gone on to develop a series of BN models, culminating in the AID tool [14], the MODIST models [15], and the extensive trials of revised models at Philips [16]. A similar model has been developed by Siemens [35]. Those models were used to provide improved methods of risk assessment for project managers, with special emphasis on defect predictions and effort prediction.

Several other groups have also researched the use of BN-based software process models. Wooff et al. [18] have developed BNs modeling the software test process, while Stamelos et al. [36] used COCOMO81 cost factors to build a BN model of software project productivity. Bibi and Stamelos [17] have shown how BNs can be constructed to model IBM's Rational Unified Process.

While these models can be adapted to agile development processes, they are not specifically targeted at such environments. Agile methods, such as XP, are characterized by highly iterative approaches to software development. If each iteration is treated as if it were a miniproject in its own right, then existing models would quickly result in BNs that are unmanageable, laborious to maintain, and computationally infeasible. What is needed is an extremely small core model that can be extended as needed.

3 DEFINITIONS AND TERMINOLOGY

The basic unit of work in XP is the *User Story*. When an XP iteration finishes, the estimated efforts for the completed user stories are added together to create the PV. In the sections that follow, we describe how user stories and PV are defined and how they are incorporated into the model.

3.1 User Stories

Developers assign the effort that they believe is required for them to design, code, and test each user story. Efforts are estimated using a unit called *Ideal Engineering Days* (IEDs). This is a day devoted entirely to user story completion, free from overheads and distractions. It includes detailed design, coding, unit testing, and acceptance testing. It excludes all other effort that can consume developers' time, including but not limited to administrative tasks, mentoring, support, and learning.

We denote the estimated effort for the j th user story in iteration i by U_i^j .

3.2 Project Velocity

Once iteration i is complete, the estimates for the completed user stories are added together. This is the PV V_i for iteration i , i.e.,

$$V_i = \sum_{j \text{ completed in } i} U_i^j. \quad (1)$$

Assuming that the next iteration, $i + 1$, is of the same length, the customer selects the highest priority uncompleted user stories whose estimated IEDs sum to V_i . These user stories are then scheduled for iteration $i + 1$. The work scheduled for iteration $i + 1$ therefore has the same estimated ideal effort as the estimates for the actual work completed in iteration i .

Note that the actual effort to complete a user story is not used here. To relate actual productive effort to estimated productive effort (i.e., PV), we introduce a bias, b_i , into the model. Note that the word "bias" is not intended in the statistical sense of a biased estimator.

If A_i^j are the actual efforts taken, then

$$b_i = \frac{\sum_j U_i^j}{\sum_j A_i^j} = \frac{V_i}{\sum_j A_i^j}. \quad (2)$$

3.3 Process Factors

To model the relationship between total effort and PV, there is a single controlling factor that we call Process Effectiveness, e . This is a real number in the range $[0, 1]$. A Process Effectiveness of one means that all available effort becomes part of the productive effort.

The Process Effectiveness is, in turn, controlled by two further parameters: Effectiveness Limit, l , and Process Improvement, r . The Process Improvement is the amount by which the Process Effectiveness increases from one XP iteration to the next. To allow for failing projects, the Process Improvement can take on negative values.

The Effectiveness Limit recognizes the fact that there are often limits to how productive a team of people can be. Every software development process has overheads. Even when all team members are working at their peak efficiency, these overheads prevent them from applying 100 percent of the available effort to delivering new functionality. And even though the team's productivity may improve over time, it is always bounded by this process-dependent limit. Effectiveness Limit is therefore the maximum value that the model allows Process Effectiveness to take.

Note that all of this relies on minimal assumptions: Effort either contributes to delivered functionality or it does not. The ratio between productive effort and total effort exists whether we call it Process Effectiveness or not. This ratio varies between iterations and has a limit, even if the limit is unity. As the core model contains variables based only on these factors, it too is based upon minimal assumptions.

4 BAYESIAN NET MODEL

The BN used to model PV is shown in Fig. 2. Table 1 summarizes the model variables for the BN. Measures of effort are denoted by capital letters. All other variables use

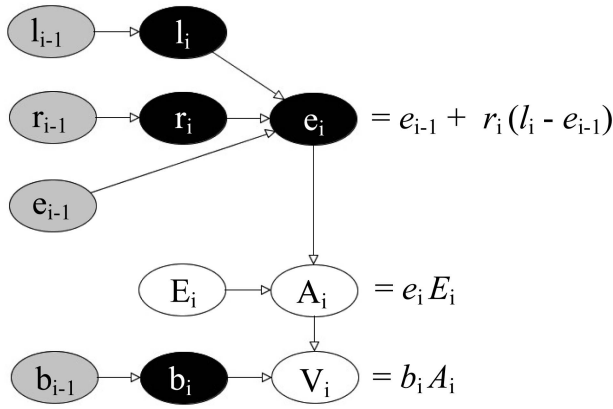


Fig. 2. Project velocity model.

lower case letters. Subscripts are used to denote a specific XP iteration. For example, V_2 denotes the velocity in iteration 2. Where the iteration is not important, we drop the subscript and refer simply to V .

When we wish to distinguish between a model prediction and a measured value, we will use an underscore to denote the measurement. So, if V_3 is the predicted value for the velocity at iteration 3, then $\underline{V}_3$ is the measured value.

Not all of the variables shown in Table 1 are shown in Fig. 2. Several of the variables are included only to make the definitions of others more rigorous (d and p). Some exist to relate the model to XP concepts (f and U) and others to relate the model to management concepts (s). Only E and V are ever entered into the model. The variables V and s are the outputs.

Before presenting the model in detail, we need to discuss a few preliminaries about DBNs.

4.1 Dynamic Bayesian Networks

DBNs extend BNs by adding a temporal dimension to the model. Formally, a DBN is a temporal model representing a dynamic system, i.e., it is the system being modeled, which is changing over time, not the structure of the network [9]. A DBN consists of a sequence of identical BNs, $\mathbf{Z}_t$, $t = 1, 2, \dots$, where each $\mathbf{Z}_t$ represents a snapshot of the process being modeled at time t . We refer to each $\mathbf{Z}_t$ as a *timeslice*. For XP, where the software production process is split into a series of discrete iterations, this is a particularly apt approach.

The models presented here are all first-order Markov. This means that $P(\mathbf{Z}_t | \mathbf{Z}_{1:t-1}) = P(\mathbf{Z}_t | \mathbf{Z}_{t-1})$ (informally, the future is independent of the past given the present). The first-order Markov property reduces the number of dependencies, making it computationally feasible to construct models with larger numbers of timeslices. Consistent propagation is achieved using standard junction tree algorithms [6]. These algorithms provide exact (as opposed to approximate) propagation in discrete BNs and are generally regarded as among the most efficient BN propagation algorithms [31].

Nodes that contain links between two timeslices are referred to as link nodes. Fig. 2 shows a single timeslice $\mathbf{Z}_t$, $t = 1, 2, \dots$, but with the link nodes from the previous timeslice shown lightly shaded. The link nodes to the next

TABLE 1
Symbol Definitions

Symbol	Meaning
d_i	Number of working days in iteration i , $d_i = 0, 1, 2, \dots$. This is an integer value.
p_i	Number of team members in iteration i . This can be fractional if one or more people do not work full time on the project, $p_i \in [0, \infty)$.
E_i	Iteration effort in man-days, $E_i = p_i \times d_i$, $E_i \in [0, \infty)$.
e_i	Process effectiveness in iteration i , $e_i \in [0, 1]$.
l_i	Effectiveness limit. The maximum value that the e_i can take, $l_i \in [0, 1]$.
r_i	Process improvement, $e_i = e_{i-1} + r_i \times (l_i - e_{i-1})$, $r_i \in [-1, 1]$.
A_i	Actual productive effort in iteration i , $A_i = E_i \times e_i$, $A_i \in [0, \infty)$.
b_i	Estimation bias, $b_i \in [0, \infty)$.
V_i	Predicted PV, $V_i = A_i \times b_i$.
U_i^j	Estimated effort of j^{th} user story in iteration i , $U_i^j \in [0, \infty)$.
$\underline{V}_i$	Measured PV in iteration i , $V_i = \sum_j U_i^j$, $V_i \in [0, \infty)$.
s_i	Productive effort to date, $s_i = s_{i-1} + V_i = \sum V_i$, $s_i \in [0, \infty)$. Predicted PV, V_i , is replaced by measured PV, $\underline{V}_i$, when an iteration has completed.
f_i	Load Factor in iteration i , $f_i = E_i / V_i$, $f_i \in [1, 5]$. Used to estimate timescales. This is often used in XP planning but is not used in the model. The upper limit is arbitrary.

timeslice are shaded black. Fig. 3 shows the same model, this time “rolled out” as a two-iteration DBN (link nodes are shaded).

The models in this paper were implemented using the AgenaRisk toolset [3]. This was due, among other things, to its ability to build dynamic models, to handle continuous variables, and the availability of a wide range of built-in conditional probability functions. Readers wishing to run the model described in this paper for themselves can do so

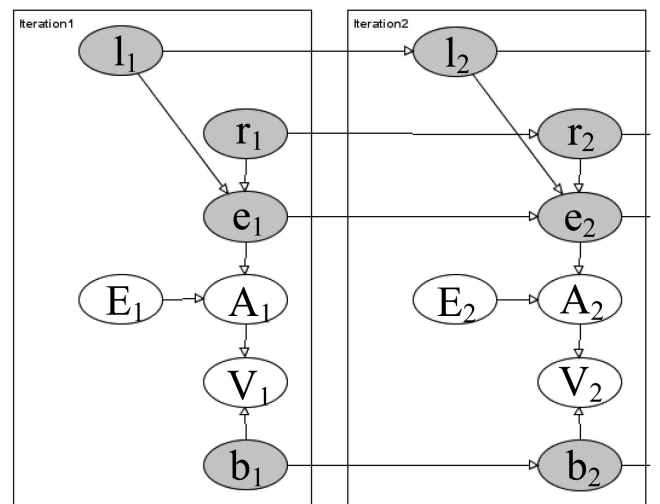


Fig. 3. Model as a DBN.

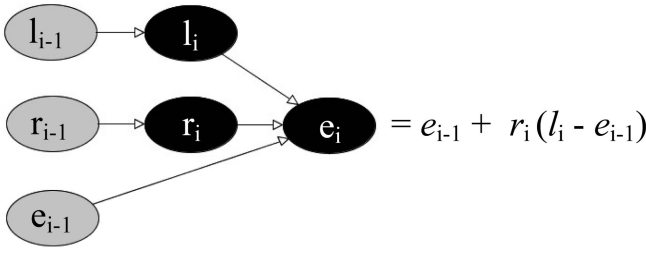


Fig. 4. Fragment 1—Process effectiveness nodes.

by downloading AgenaRisk from [3] (free but time-limited) and the model from [4].

4.2 Parameter Learning

The process effectiveness limit (l_i), rate of process improvement (r_i), and bias (b_i) are the key parameters in this model. Between them, they control the process effectiveness node, which in turn controls the velocity node. It is important that the model is capable of adjusting these parameters as a result of entering data about the project. In particular, the model must respond to observations of the $\underline{V}_i$.

4.3 Iteration Model

The BN shown in Fig. 2 is used as a single iteration model for PV. The model is best thought of as comprising three distinct fragments. In the sections that follow, links between the fragments are not shown. Refer to Fig. 2 to see how the fragments are connected.

Fragment 1 controls the Productive Effort (Fig. 4). A single variable, Process Effectiveness (e_i), is assumed to determine the Productive Effort. High Process Effectiveness means a high Productive Effort and a correspondingly high velocity. Process Effectiveness increases or decreases based on the value of the Process Improvement (r_i). It is constrained to the range $[0, l_i]$.

The CPD of l_i is a function of l_{i-1} . In this case, l_i is set equal to l_{i-1} . The process effectiveness limit (l_i) is really a single variable, which is global to all timeslices. Copying it between timeslices allows us to preserve the first-order Markov property. Similarly, r_i is just a copy of r_{i-1} .

Fragment 2 contains the “effort” nodes (Fig. 5). It combines the total Iteration Effort (E_i) with the process effectiveness (e_i) to create the actual Productive Effort (A_i). Note that, although A_i is not required by the XP methodology, we need it in this model for reasons that will be explained below. We do not expect A to be observed in real projects.

Fragment 3 holds the PV (Fig. 6). Velocity can either be predicted by the model (V_i) or, once an iteration is completed, it can be entered as evidence ($\underline{V}_i$) and used to learn the model parameters. The bias, b_i , allows for any consistent bias in the team’s effort estimation. If there was no bias, then the productive effort, A , would be the same as V and there would be no need to distinguish between the two.



Fig. 5. Fragment 2—Effort nodes.

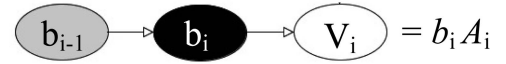


Fig. 6. Fragment 3—Project velocity.

4.4 Setting the Initial Conditions

An initial timeslice, iteration 0 (shown in Fig. 7), is used to set the initial model conditions.

For iteration 0, the prior distributions of the input effectiveness limit (l_0), process improvement (r_0), and process effectiveness (e_0) are all set to be normal distributions with a standard deviation of 0.3 and means of 0.8, 0.2, and 0.3, respectively. These values are based on a controlled case study by Abrahamsson and Koskela [8], where process effectiveness varied between 0.4 and 0.75. We have simply extended this range slightly and chosen r_0 so that the lowest to highest transition can take place within four iterations.

The prior of the estimation bias (b_0) is set to a lognormal distribution with a mean of approximately 1.0 and a variance of 0.1. The lognormal distribution follows from the fact that the bias cannot be less than zero but has no upper bound. For example, a pessimistic bias, where estimates are twice the actual, results in a bias of 2, whereas an optimistic bias results in a bias of 0.5. This distribution is confirmed empirically, for example, by Little [13].

The estimation bias reflects the cumulative bias of the whole team. In order to significantly deviate from a value of one, there must be a *systematic* tendency for the team to overestimate or underestimate the size of user stories (for example, due to a different choice of measurement units). As such, the variance of this node is not directly related to the estimation bias of individuals, as reported by Jørgensen et al. [39], [40], [41]. The variance has therefore been chosen to be considerably smaller than typical values reported by Jørgensen et al..

The choice of these priors is discussed further in Section 7.

Evidence is entered in all the E_i nodes so the prior distributions of these nodes have no effect.

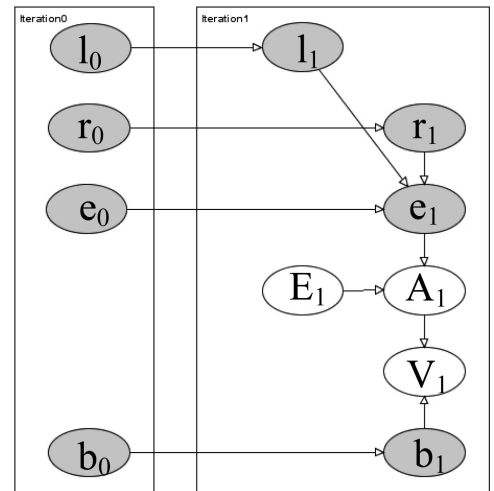


Fig. 7. Initial velocity model.

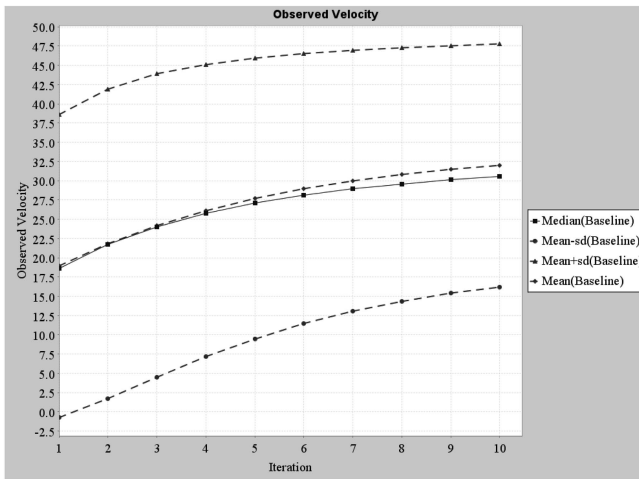


Fig. 8. Project velocity values—median, mean, mean ± 1 standard deviation.

5 MODEL BEHAVIOR

Fig. 8 shows the predicted values of the PV for a hypothetical project with 10 iterations and 50 h of effort available in each iteration (i.e., $E_i = 50$, $i = 1, \dots, 10$). The central dotted line is the mean, with the outer dotted lines showing ± 1 standard deviation. The solid line is the median value. This is based solely on the model's initial conditions.

The Process Effectiveness increases with each iteration by an amount equal to the Process Improvement. It flattens out as it begins to hit the Effectiveness Limit. As can be seen from the graph, this leads to the PV starting fairly low and gradually increasing with each iteration. Being able to model and predict this type of behavior was one of the main objectives of the core model.

This is our "Baseline" scenario, with no PV evidence entered into the model. By entering PV evidence, we can construct various alternative scenarios and compare the learned parameters and predicted values of future PV. The values shown in Table 2 were used to construct three such scenarios, all based on 50 h of available effort per iteration. No values were entered for V_9 or V_{10} , allowing the model to predict these values. These represent projects that are, respectively, failing, performing as expected, or progressing with great success. We refer to these as the "Failing," "Average," and "Success" scenarios, respectively.

Note that the "Success" scenario uses deliberately unrealistic figures in order to test the range of the model.

TABLE 2
PV Values for Three Scenarios

PV Scenario	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8
Failing	2	3	3	4	4	3	4	4
Average	20	25	27	28	28	29	30	31
Successful	200	205	210	215	219	223	225	227

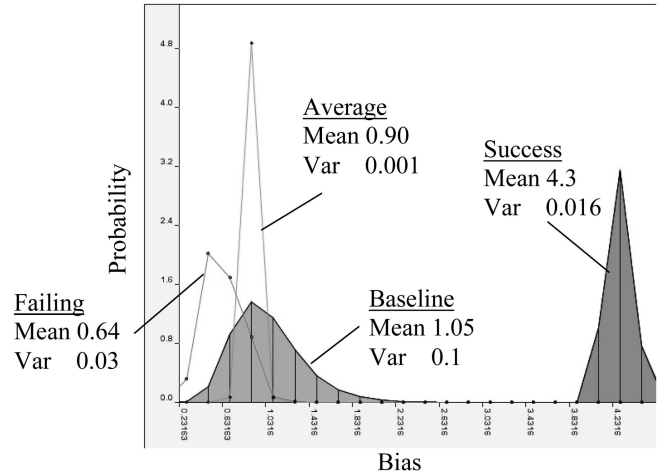


Fig. 9. Bias distribution, iteration 10.

5.1 Parameter Learning in Different Scenarios

Fig. 9 shows the resulting distributions of the bias node, b_{10} . There are four distributions, one for each scenario. The "Failing," "Average," and "Baseline" scenarios have mean values close to one, although both the Failing and Average scenarios have reduced variances compared to the baseline. The reduced variances are to be expected from scenarios where evidence has been entered.

In Fig. 8, the Baseline scenario predicted values for V_1 to V_8 in the range of 18–30. However, the Success scenario entered evidence in the range of 200–227, indicating that the project team has done 200–227 estimated IEDs in a single iteration with only 50 man-days of effort. Clearly, this can only come about if their estimates are significantly biased, and indeed, the model suggests that the bias in this case has a mean value of 4.3. This only accounts for part of the high PV values however. The remainder is accounted for by an increased effectiveness limit (Fig. 10), which allows a greater process effectiveness.

As we might expect, the Failing scenario shows a poor effectiveness limit and a very small improvement in process

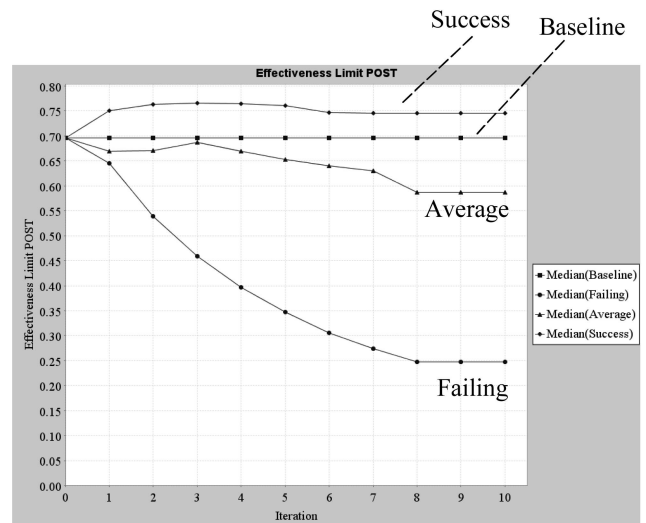


Fig. 10. Effectiveness limit, median, 10 iterations.

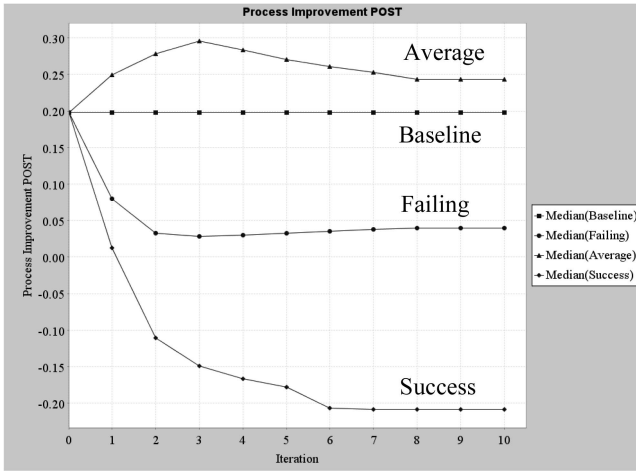


Fig. 11. Process improvement, median, 10 iterations.

effectiveness (Fig. 11). Surprisingly, the success scenario shows an even worse process improvement. However, this is because the model is forced to assume a very high process effectiveness in the initial iterations. The values provided are so far outside the normally expected range that the model is continually trying to compensate by bringing the process effectiveness back down again. By iteration 6, the process improvement finally begins to stabilize.

Both the Effectiveness Limit (Fig. 10) and the Process Improvement (Fig. 11) change as evidence is entered in the first eight iterations. The model therefore learns as new evidence is entered and changes its predictions accordingly.

Fig. 12 shows the behavior of the Bias node, b_i , in the Average scenario. The central dotted line, which is almost co-incident with the solid line, shows the mean and median values, respectively. The outer dotted lines show the mean ± 1 standard deviation. The standard deviation gets smaller as more evidence is entered into the model. This illustrates that not only does the model learn the values of its parameters, but the uncertainty in those values decreases as more evidence becomes available.

5.2 Indicator Nodes

Indicator nodes are nodes with a single parent and no children. They are often used to provide evidence for variables that are themselves unobservable. Indicator nodes

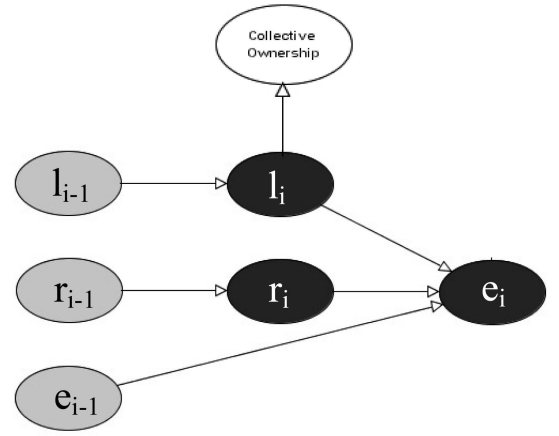


Fig. 13. The "Collective Ownership" indicator node.

are one of the main mechanisms used to introduce XP practices into the model.

XP practices cannot be categorized as simply being "implemented" or "not implemented." There are degrees to which various practices are adopted. For example, a team may choose to program in pairs for complex parts of the code and program individually when writing routine code. It is important, therefore, that XP practices are represented by nodes with a sufficient range of states to reflect the degree of variation of that practice within the project.

An indicator node for the Effectiveness Limit is shown in Fig. 13: the "Collective Ownership" node. This is the extent to which collective code ownership is practiced. (The node is added for illustration purposes only, we make no claims as to the real effect of changes in this particular XP practice.) It is a ranked node, consisting of five discrete values ranging from Very Low to Very High. Ranked nodes allow the user to enter a range of values for "Collective Ownership." The probability of these five values is derived from a truncated normal distribution whose mean is l_i and whose variance is arbitrarily set to 0.1. The variance determines the strength of the relationship—a small variance makes the nodes highly dependent, a large variance makes the marginal distribution of the indicator closer to being uniform. This distribution ensures that a high degree of collective ownership leads to a high effectiveness limit. More information on ranked nodes and the use of the truncated normal distribution can be found in [12].

With no evidence, the node plays little part in the model and its parent, l_i , remains constant from one iteration to the next (the "Baseline" scenario). However, when we set the value of "Collective Ownership" in each iteration to "Very High" (the "High" scenario), then the situation changes. The evidence back propagates to l_i . Because of the learning mechanism described above, the effect is cumulative and the mean value increases across iterations. The difference is shown in Fig. 14.

Values entered into this node are examples of expert judgment. The ease with which expert judgment can be combined with objective evidence and prior assumptions is one of the benefits of the Bayesian Network approach to modeling.

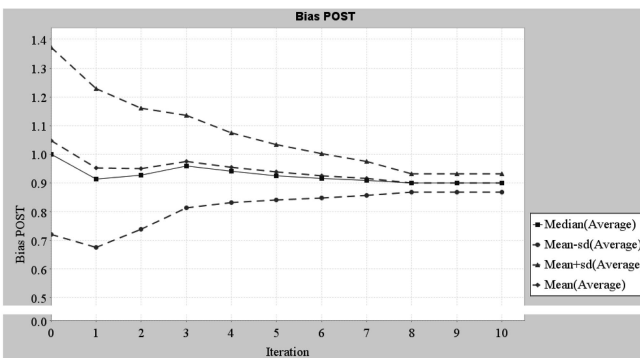


Fig. 12. Bias, Average scenario, median, mean ± 1 standard deviation.

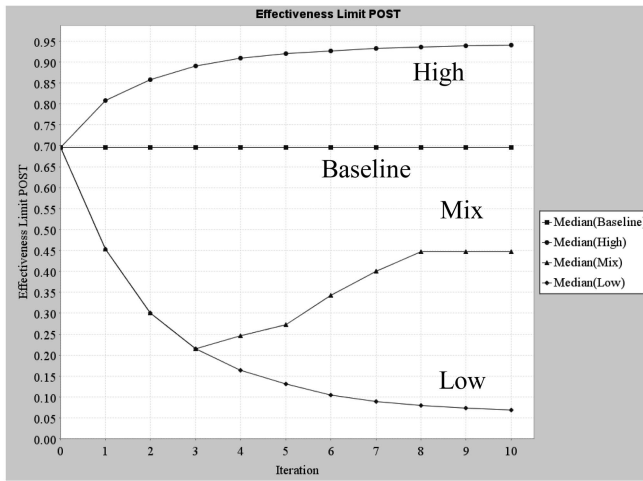


Fig. 14. Effectiveness Limit with and without indicator node evidence.

Two other scenarios are also shown, one where the Collective Ownership node is always set to “Very Low” (the “Low” scenario) and a slightly more realistic case (the “Mix” scenario). In the Mix scenario, Collective Ownership starts off “Very Low.” However, the management realizes that there is a problem and takes steps to improve collective ownership. By iteration 4, Collective Ownership improves to “Medium,” and by iteration 6, it achieves a “High” value.

The extent to which XP practices are implemented can therefore have a dramatic effect on the model parameters, which in turn propagates through to the model’s predictions.

It is not necessary to include all XP practices as indicator nodes in all iterations. If a practice, such as pair programming, say, is consistently maintained at the same level in all iterations, then its effect will be included in the learned values of the model parameters. Only practices that affect project velocity and which vary significantly between iterations need to be included as indicator nodes.

The direction of the arcs in indicator nodes can often be counterintuitive. For example, it could be argued that Collective Ownership is really a causal factor of the Effectiveness Limit and that as such the arrow should be in the opposite direction. However, due to Bayes’ theorem, we can always construct a CPD that allows us to reverse the direction of the arrow. We do this when it is easier to obtain one CPD than another. In the case of the Effectiveness Limit, modeling multiple XP practices as causal factors would require us to construct a CPD that was conditional upon all possible combinations of those causal factors, something that is, for all practical purposes, impossible. We have therefore chosen to model XP practices as indicator nodes.

As a result, XP practices become dependent on one another due to their common parent. However, this does not present a problem. We are only interested in *changes* to XP practices. It is only when the practices change that we enter evidence into their corresponding indicator nodes. If they are left without evidence, then they have no effect on the model. If one of them (without evidence) is affected by another (with evidence), then we simply ignore it—the aim of the model is not to predict which XP practices are being used.

TABLE 3
Motorola Project Data

i	1	2	3	4	5	6	7	8
d_i	15	15	15	16	12	10	8	10
p_i	3	3	6	6	7	7	9	4
E_i	45	45	90	96	84	70	72	40
$\underline{V}_i$	9	13	35	30	40	40	36	20

6 MODEL VALIDATION

We apply the model to an industrial case study (Section 6.1). The model can learn from the initial data entered from the project (Section 6.2) and adjusts its predictions once beneficial XP practices are taken into account (Section 6.3). Section 6.4 provides an example of how the model can be calibrated for a specific XP practice. Finally, in Section 6.5, the model provides predictions for the time taken to deliver a fixed amount of functionality. These are in good agreement with the actual functionality delivered.

6.1 The Motorola Project

Williams et al. [5] provided a detailed description of an XP project developed at Motorola. The project was developed in a series of eight iterations of between two and three week durations. The number of people on the team varied from three to nine over the duration of the project. The full data set is shown in Table 3.

The definition of PV used by the Motorola team corresponds to what we have called Process Effectiveness. We will continue to use the definition given in (1). The values for $\underline{V}_i$ given in Table 3 have been calculated using our definition.

Initially, we simply enter values for E_i into the model (no values for $\underline{V}_i$ entered). Fig. 15 shows the resulting marginal distributions, which are generated for the V_i node. There is one distribution for the node in each timeslice.

The median values from the V_i distributions are shown in Fig. 16 (the “Predicted” graph). Actual values for $\underline{V}_i$ are shown in the same figure for comparison (the “Actual”

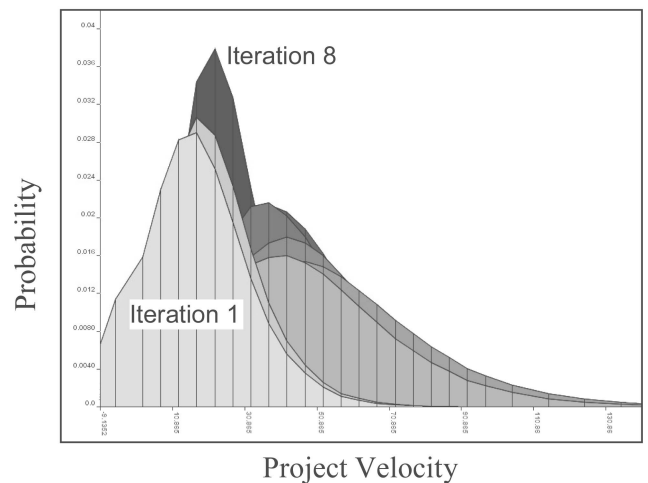


Fig. 15. Distributions for V_i , one per timeslice.

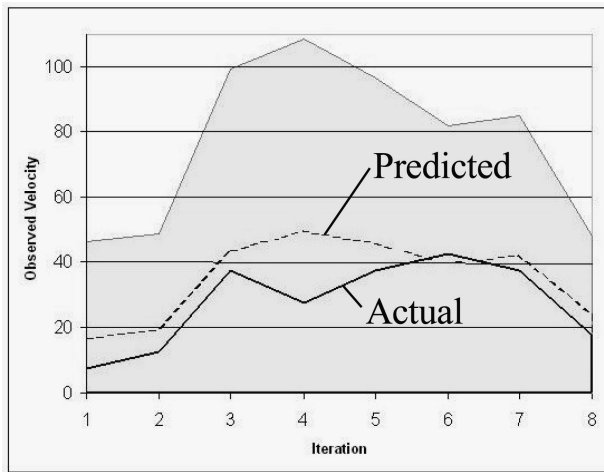


Fig. 16. Predicted versus actual Motorola V (medians). Actual values are bold, predicted values are dashed. The shaded area shows predicted medians ± 2 standard deviations.

graph). The large “Actual” dip in iteration 4 is put down to a post-Christmas malaise by the Motorola team.

Fig. 16 also shows a 2-standard deviation envelope around the predicted values (excluding meaningless negative values). The envelope is extremely wide at this stage. This is as we would expect. The model is making predictions purely on the basis of empirical industry priors and available effort values. With no indication as to the productivity of the development team, the uncertainty surrounding the predictions can be very high.

6.2 Parameter Learning

There are a number of problems with the predicted values in Fig. 16. The most obvious is that, apart from iteration 6, the predicted values are consistently too high. In this section, we demonstrate how the model can learn from real project data and quickly improve the accuracy of its predictions.

The effect of this learning process can be seen by taking the “Predicted” scenario and entering V_i observations for completed iterations. As each new piece of information is entered, propagation takes place, causing the distributions for the model parameters to be updated. These updated parameter distributions then affect the predictions of future iterations.

The graphs in Fig. 17 show the change in predicted values when V_1 and V_2 have been entered. The whole of the “Predicted” graph moves to lower values as the model learns from the observations. The predictions for V_3 and V_4 improve as a result.

As expected, the uncertainty surrounding the predictions narrows considerably, reflecting the fact that the model has updated its expectation about team productivity. However, although the actual values remain well within the 2 standard deviation envelope, in absolute terms the predicted values for V_5 , V_6 , and V_7 are significantly worse.

The paper by Williams et al. [5] points out that various XP practices were implemented more effectively in later iterations. In Section 6.3, we show how this can be incorporated into the model.

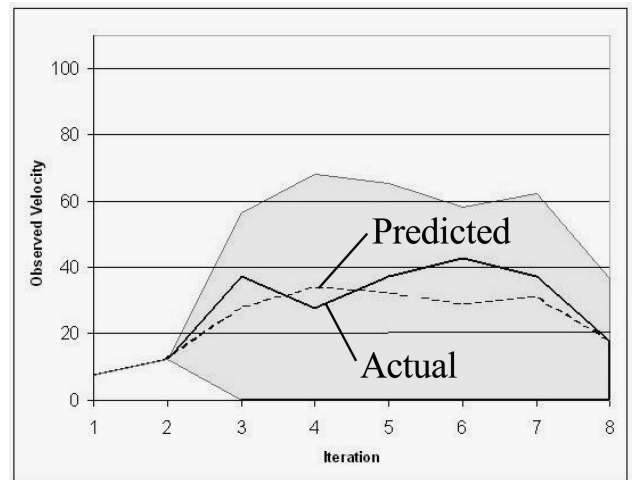


Fig. 17. Predicted and actual V , two observations. Actual values are bold, predicted values are dashed. The shaded area shows predicted medians ± 2 standard deviations.

6.3 “Onsite Customer” as an Indicator Node

An indicator node for the Effectiveness Limit is shown in Fig. 18: the “Onsite Customer” node. This is the extent to which an authoritative customer was available to answer questions about requirements and provide feedback on development. It is a ranked node, consisting of five discrete values ranging from Very Low to Very High. These discrete values define five equal, discrete partitions of the real number range $[0, 1]$.

The probability of these five values is derived from a truncated normal distribution whose mean is l_i and whose variance is set to 0.1. This distribution ensures that a high degree of customer input leads to a high effectiveness limit.

It is important to emphasize that the values entered into the “Onsite Customer” node must be relative to the need for customer input. If the project team have developed similar projects for this customer in the past or are themselves experts in the application domain, then constant customer input may not be useful. In these circumstances, a “Very High” value for “Onsite Customer” might be appropriate, even if the customer is not physically present but was still able to provide input when needed.

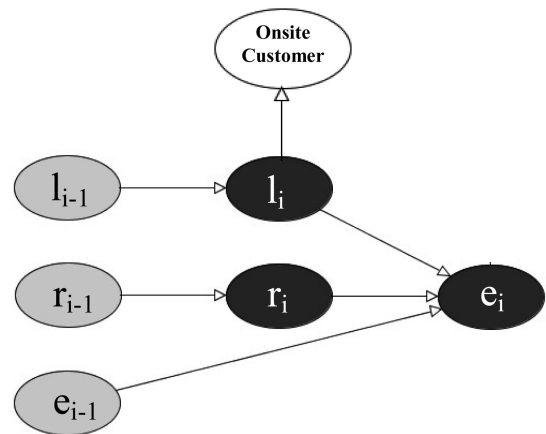


Fig. 18. The “Onsite Customer” indicator node.

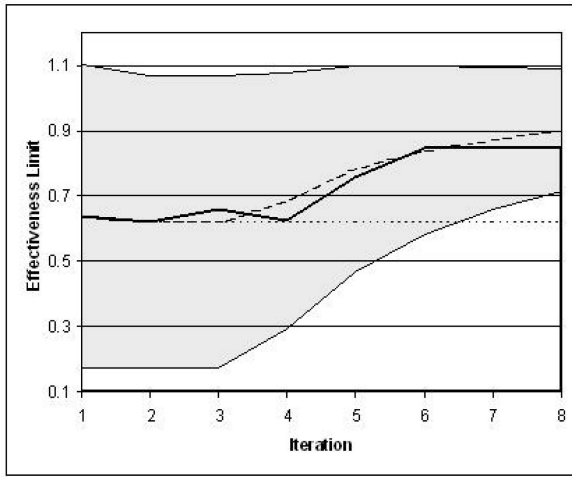


Fig. 19. Effectiveness Limit with and without indicator node evidence. Actual values are bold, predicated values (after “Onsite Customer” evidence) are dashed. The shaded area shows predicted medians ± 2 standard deviations. The dotted line shows the learned values without “Onsite Customer” evidence.

Fig. 19 shows how the indicator node’s parent is affected by changes in its values. When only effort data are entered, the median from the Effectiveness Limit node’s distribution is simply a straight line (not shown). When all the V_i data and the appropriate onsite customer data are entered, then the Effectiveness Limit varies throughout the project (the “Actual,” bold curve). The dotted, “Learned” curve shows the Effectiveness Limit that is learned when only V_1 and V_2 have been entered as observations. This is the curve that is responsible for the predictions shown in Fig. 17.

At the start of the fifth iteration, the Motorola team had constant access to an onsite customer. The “Onsite Customer” indicator node was therefore set to “Very High” for these iterations. The result is the dashed, “Onsite Cust” curve. It shares the same values for the Effectiveness limit as the “Learned” curve, until the values for the Onsite Customer indicator node are modified.

The result of entering indicator node evidence is an improvement in the predicted V_i values, as shown in Fig. 20.

6.4 Calibrating the Onsite Customer Node

The distribution for the “Onsite Customer” node is based on the data from Korkala et al. [10]. In their paper, four case studies are described with varying degrees of customer interaction. The percentage of effort devoted to fixing defects, including specification defects, varied greatly in the four case studies. Where customer input was very high, only 6 percent of the effort was spent fixing defects. Moreover, this level remained constant across iterations. At the other extreme, when customer input was very low, the time spent fixing defects grew across iterations until it reached about 40 percent in iteration 3.

Our model does not explicitly include details of defect fixing effort (including requirement defects); they are simply included as effort that does not contribute to V . We therefore make the following definitions and assumptions concerning the relationship between defect fixing effort and nonvelocity effort:

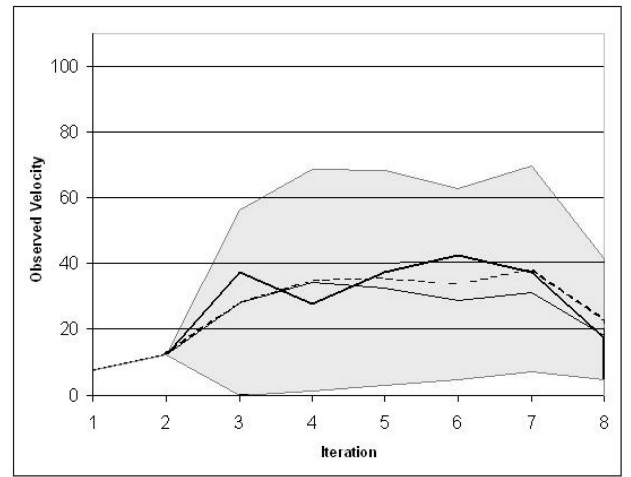


Fig. 20. V with and without Onsite Customer evidence. Actual values are bold, predicted values are dashed. The shaded area shows predicted medians ± 2 standard deviations. The solid gray curve shows the predicted values before Onsite customer evidence.

1. Define “Miscellaneous Effort,” m_i , to be the fraction of effort that does not contribute to completed user stories: $E_i = V_i + m_i$.
2. Miscellaneous effort is composed of a variable component due to defect fixing effort, d_i , and a set of fixed overheads, o_i : $m_i = d_i + o_i$. This does not provide a full description of miscellaneous effort, but it is adequate for this model.
3. When the onsite customer input is at its maximum, the rework effort is at its minimum.

With these assumptions in place, we can use the BN shown in Fig. 21 to calibrate the Onsite Customer node. The algorithm proceeds as follows:

1. An initial guess is made at the Onsite Customer distribution.
2. The values of o_i are chosen so that, when the Onsite Customer node is set to “Very High,” d_i produces a constant mean value of about 6 percent across all iterations.

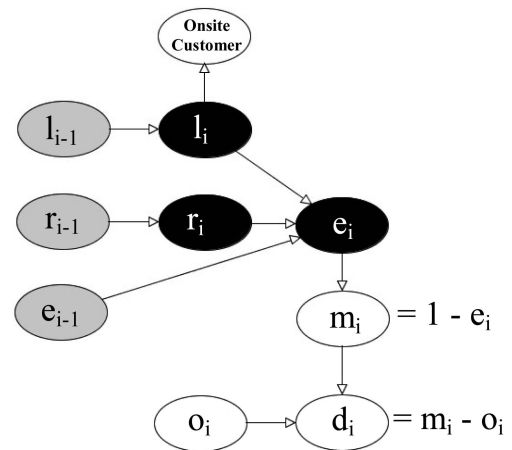


Fig. 21. BN used to calibrate the Onsite Customer node.

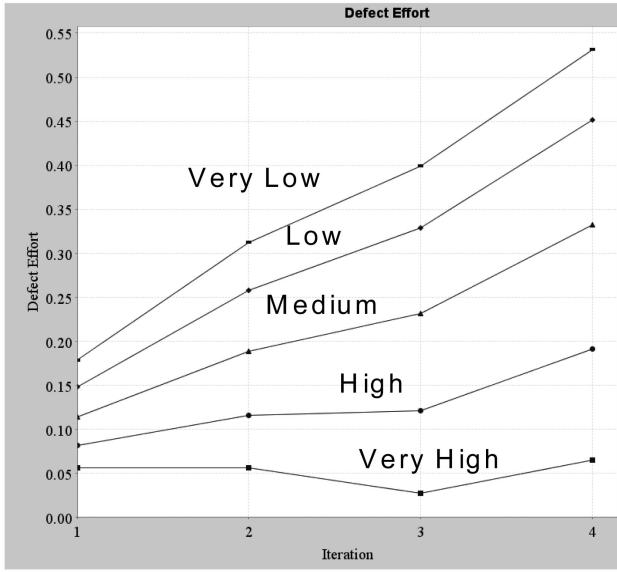


Fig. 22. Defect effort percentages for each Onsite Customer setting.

3. Modify the Onsite Customer distribution, with the value set to “Very Low” until the time spent fixing defects in iteration 3 is about 40 percent.
4. Repeat Steps 2 and 3 until both conditions are satisfied simultaneously.

The resulting defect effort percentages for each value of “Onsite Customer” across four iterations are shown in Fig. 22. These are similar to the empirical curves in [10, Fig. 3].

In the calibration of the Onsite Customer node, we have taken advantage of an independent study into the effectiveness of good customer communication [10]. In the case of other XP practices, there clearly needs to be some empirical evidence in order to calibrate the corresponding indicator nodes. This need not be in the form of a controlled case study like [10]; it can be based on the experience/opinions of industry professionals, for example. We have included the possible absence of such information as threat to the validity of the model.

6.5 Timescale Prediction

Fig. 23 shows a slightly modified version of the velocity fragment of the model. This includes an additional link node, s_i , which acts as the cumulative sum of V to date.

Plots of s_i for the initial prediction, the learned prediction (after two iterations), and the actual scenarios are shown in Fig. 24. If the total estimate to complete the project is, say, 200 IEDs, then we can immediately read off from the graph how long it will take to complete the project.

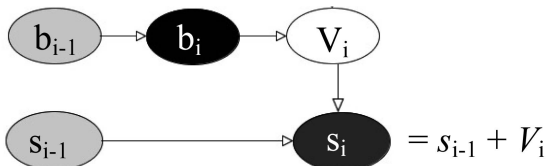


Fig. 23. Project velocity summed to date.

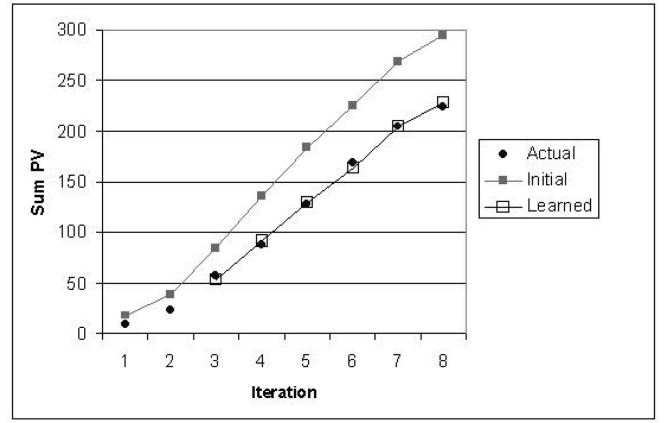


Fig. 24. s_i after each iteration.

The initial predictions of the model are too optimistic. However, once the model has learned from the V_1 and V_2 observations and account has been taken of the onsite customer, the predictions are virtually indistinguishable from the actual outcome. We can measure the Magnitude of Relative Error (MRE) for median values of s_i , defined by

$$\text{MRE}_i = \frac{|s_i - \underline{s}_i|}{\underline{s}_i}. \quad (3)$$

The model’s initial median predictions have MRE values between 0.3 and 0.96. This range reduces to 1E-5 to 0.07 after learning. The improvement is summarized in Table 4. As further evidence is entered into the model however, the accuracy of its predictions does not necessarily improve. For example, when V_3 is entered, the model has no way of knowing that the next PV will be affected by the Christmas break. It therefore predicts a much higher value than is actually achieved. This causes the model’s accuracy to degrade when V_3 is entered. We can examine this effect quantitatively by taking the Mean of the MRE values (MMRE) for the model’s median s_i predictions. After V_3 is entered, this jumps suddenly before gradually improving

TABLE 4
MRE Values for s

Iteration	s_i	s_i^0	MRE_i^0	s_i^2	MRE_i^2
1	9	17.6	0.96		
2	22.5	39.0	0.73		
3	57.9	84.6	0.46	54.1	0.07
4	87.9	136	0.55	91.4	0.04
5	127.8	184	0.44	129.0	0.01
6	168.4	225	0.34	164.4	0.02
7	204.4	269	0.32	204.4	0.00001
8	224.4	294	0.31	228.4	0.02
MMRE			0.51		0.026

The true functionality delivered after iteration i is s_i . The initial prediction for s_i is s_i^0 . The MRE for s_i^0 is MRE_i^0 . The prediction for s_i after two iterations is s_i^2 . The MRE for s_i^2 is MRE_i^2 . The Mean MREs are shown at the bottom of the relevant columns.

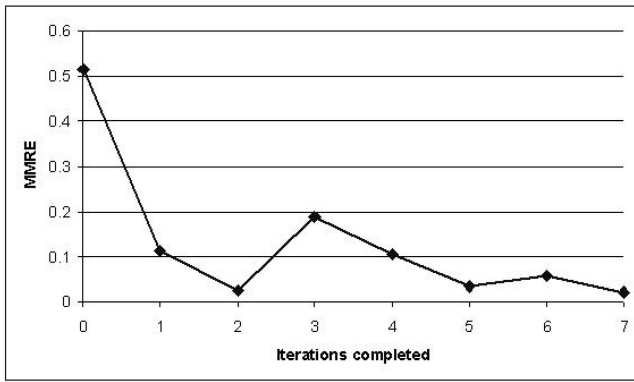


Fig. 25. Mean MRE for future s_i predictions after each iteration.

once again. This is illustrated in Fig. 25. We discuss this behavior further in Section 7.

The Motorola project completed 224 IEDs of functionality before the project ended. The model can quantify the uncertainty involved in completing 224 IEDs within eight iterations. Fig. 26 shows the cumulative distribution functions for the s_i node in iteration 8. The vertical line allows us to read off the probability of completing up to 224 IEDs by the end of the eighth iteration. For the “Initial” scenario, there is only a 25 percent chance of completing up to 200 IEDs. Once the model has learned from $\underline{V}_1$ and $\underline{V}_2$, the probability is revised up to a 65 percent probability. This means that the model was initially too optimistic in its predictions (a 65 percent chance of delivering up to 224 IEDs means a 35 percent chance of delivering *more than* 224 IEDs).

7 CONCLUSIONS AND DISCUSSION

We have developed a model of XP PV and shown that it reproduces known empirical behavior from iterative projects.

The model has been applied to a real industrial project. Incorporating data from the early part of the project enabled the model to update its parameters and improve its predictions. When this was combined with knowledge about the presence of an onsite customer, the model was able to make extremely accurate predictions about the level of functionality delivered over time. Other XP practices can be incorporated in the model using similar techniques.

While the model presented here has successfully demonstrated the benefits of using a learning BN model in XP projects, we recognize that there are a number of threats to its validity:

1. The model relies on having sufficient degrees of freedom to learn from its environment. This is principally accomplished by updating the parameter nodes l_i and r_i . It is possible that those are insufficient to accommodate the full range of behaviors of real XP projects or that some future XP practices cannot be wholly accommodated as indicators of one of these nodes.
2. Only a single industrial test case has been used. Greater confidence in the model will be achieved through exposure to a greater variety of data sets.

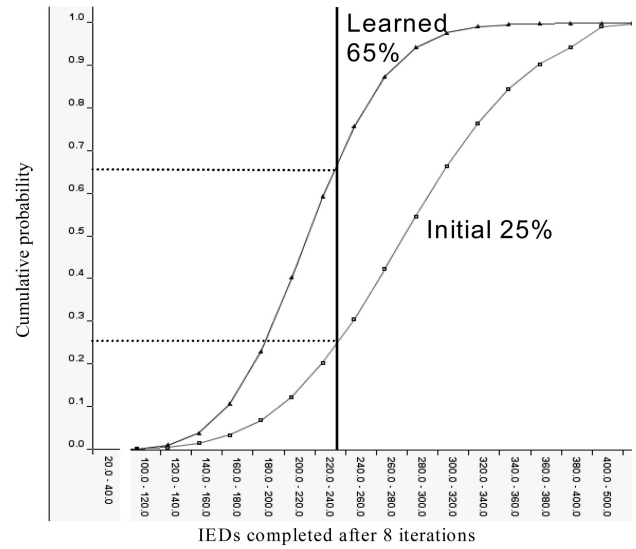


Fig. 26. Iteration 8 cumulative distributions.

3. The example shown had the benefit of real effort data from a completed project. At the start of a project, only projections of available effort are available.
4. No sensitivity analysis has been performed on the model priors in iteration 0. This is not an especially serious concern because, regardless of the initial values, the model will adapt to the current project's local conditions as soon as the first few iterations are completed. Clearly, any change in the means or standard deviations of the priors will affect the model's initial predictions. We would expect that more mature software development organizations would replace the supplied values with distributions based on their own previous metrics programs.
5. Two XP practices have been included in the model: “Collective Ownership,” using hypothetical data, and “Onsite customer,” using data from a single study. Empirical data on the effectiveness of other XP practices need to be used in order to calibrate appropriate indicator nodes.

Despite these concerns, there are a number of clear benefits to this approach:

1. Although prior metrics information is valuable, an extensive data collection phase is not essential. The model starts off making generic predictions but quickly alters them as local data become available. Developers tasked with metrics collection therefore see an immediate benefit from doing so: Predictions about their own project will improve as a result. Contrast this with traditional metrics collection programs, which often founder because of the need for long-term commitment.
2. Empirical data, project data, prior assumptions, and expert judgment are combined in a single intuitive, causal model.
3. The predictions provide probability distributions, not just single values. This makes it possible to assess the uncertainty associated with each outcome.

4. Provided suitable empirical evidence is available, it is relatively simple to add new XP practices or other environmental features, making the model extremely versatile.

The model presented here differs from many of the other causal models described in Section 2.2. Rather than trying to construct a complex graph of causal relationships, it opts instead for a very simple structure. This model recognizes that, for a large variety of reasons, software productivity varies throughout the iterations of an agile project. It therefore learns the cumulative effect of these variations rather than trying to model their interactions explicitly.

The model's simplicity does not prevent causal factors being modeled however. We have shown how changes in XP practices can be factored in. Other changes in the environment, such as the post-Christmas malaise, could also be included if this was seen to be typical. We chose not to do this, relying instead on the ability of BNs to deal with uncertainty. This enables the model to accept evidence that is considerably different from its expectations. The post-Christmas malaise caused the model to (wrongly) revise its future predictions, but it quickly recovered as more evidence became available.

Users of the model need to provide three items of information:

1. Available effort over the timescale of the project. These figures can be arrived at using traditional techniques. In the future, determining these figures can become part of the model.
2. Measured PV as it becomes available. These values are collected in almost all XP projects.
3. The extent to which XP practices are varying between iterations. These can be purely subjective judgments.

The burden on developers and managers in maintaining this model is therefore minimal. In return for this small overhead, projects get improved PV predictions, such as in Fig. 24, and a quantitative assessment of the risk, as in Fig. 26.

A similar approach can be used to create a defect prediction model, with the effort model as one of its primary inputs. This allows a family of models to be constructed, which represents a wide variety of XP environments and which can be used to model either effort alone, effort plus defects, or cost versus time trade-offs.

ACKNOWLEDGMENTS

This research was funded through the eXdecide Project (EPSRC Grant EP/C005406/1), with an additional student CASE Award supplied by Agena. The authors would like to thank the anonymous reviewers for their many constructive comments on this paper.

REFERENCES

- [1] F.V. Jensen, *Bayesian Networks and Decision Graphs*. Springer-Verlag, 2001.
- [2] F.P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*, second ed. Addison Wesley, 1995.
- [3] *AgenaRisk User Manual*, Agena, www.agenarisk.com, 2008.
- [4] http://www.dcs.qmul.ac.uk/research/radar/Projects/exdecide/xp_model.html, 2007.
- [5] L. Williams, A. Shukla, and A.I. Antón, "An Initial Exploration of the Relationship between Pair Programming and Brooks' Law," *Proc. Agile Development Conf.*, 2004.
- [6] S.L. Lauritzen and D.J. Spiegelhalter, "Local Computations with Probabilities on Graphical Structures and Their Application to Expert Systems (with Discussion)," *J. Royal Statistical Soc. Series B*, vol. 50, no. 2, pp. 157-224, 1988.
- [7] A. Ahmed, M.M. Fraz, and F.A. Zahid, "Some Results of Experimentation with Extreme Programming Paradigm," *Proc. Seventh Int'l Multi Topic Conf.*, pp. 387-390, 2003.
- [8] P. Abrahamsson and J. Koskela, "Extreme Programming: A Survey of Empirical Data from a Controlled Case Study," *Proc. Int'l Symp. Empirical Software Eng.*, pp. 73-82, 2004.
- [9] K.P. Murphy, "Dynamic Bayesian Networks: Representation, Inference and Learning," PhD thesis, Univ. of California, Berkeley, 2002.
- [10] M. Korkala, P. Abrahamsson, and P. Kyllönen, "A Case Study on the Impact of Customer Communication on Defects in Agile Software Development," *Proc. AGILE Conf.*, 2006.
- [11] N.E. Fenton and M. Neil, "A Critique of Software Defect Prediction Models," *IEEE Trans. Software Eng.*, vol. 25, no. 4, pp. 675-689, July/Aug. 1999.
- [12] N.E. Fenton, M. Neil, and J.G. Caballero, "Using Ranked Nodes to Model Qualitative Judgements in Bayesian Networks," *IEEE Trans. Knowledge and Data Eng.*, vol. 19, no. 10, pp. 1420-1432, Oct. 2007.
- [13] T. Little, "Schedule Estimation and Uncertainty Surrounding the Cone of Uncertainty," *IEEE Software*, May/June 2006.
- [14] M. Neil, P. Krause, and N.E. Fenton, *Software Quality Prediction Using Bayesian Networks in Software Engineering with Computational Intelligence*, T.M. Khoshgoftaar, ed., Chapter 6. Kluwer, 2003.
- [15] N.E. Fenton, W. Marsh, M. Neil, P. Cates, S. Forey, and T. Tailor, "Making Resource Decisions for Software Projects," *Proc. 26th Int'l Conf. Software Eng.*, 2004. ISBN 0-7695-2163-0, 397-406.
- [16] M. Neil and P. Fenton, "Improved Software Defect Prediction," *Proc. 10th European SEPG Conf.*, 2005.
- [17] S. Bibi and I. Stamelos, "Software Process Modeling with Bayesian Belief Networks," *Proc. 10th Int'l Software Metrics Symp.*, Sept. 2004.
- [18] D.A. Wooff, M. Goldstein, and F.P.A. Coolen, "Bayesian Graphical Models for Software Testing," *IEEE Trans. Software Eng.*, vol. 28, no. 5, pp. 510-525, May 2002.
- [19] L. Williams and H. Erdogmus, "On the Economic Feasibility of Pair Programming," *Proc. ICSE Int'l Workshop Economics-Driven Software Eng. Research*, May 2002.
- [20] S.A. Ross, *Fundamentals of Corporate Finance*. Irwin/McGraw-Hill, 1996.
- [21] K. Beck and A. Andres, *Extreme Programming Explained, Embrace Change*, second ed. Addison-Wesley Professional, Nov. 2004.
- [22] K. Beck, *Extreme Programming Explained, Embrace Change*, first ed. Addison-Wesley Professional, 2000.
- [23] R. Jeffries, A. Anderson, and C. Hendrickson, *Extreme Programming Installed*, first ed. Addison-Wesley Professional, 2001.
- [24] A. Cockburn and L. Williams, "The Costs and Benefits of Pair Programming," *Proc. Extreme Programming and Flexible Processes in Software Eng.*, June 2000.
- [25] J. Nosek, "The Case for Collaborative Programming," *Comm. ACM*, vol. 41, no. 3, pp. 105-108, Mar. 1998.
- [26] L. Williams, R. Kessler, W. Cunningham, and R. Jeffries, "Strengthening the Case for Pair-Programming," *IEEE Software*, pp. 19-25, July/Aug. 2000.
- [27] F. Padberg and M. Müller, "Analyzing the Cost and Benefit of Pair Programming," *Proc. Ninth Int'l Software Metrics Symp.*, 2003.
- [28] V. Misic, H. Gevaert, and M. Rennie, "Extreme Dynamics: Modelling the Extreme Programming Software Development Process," *Proc. Workshop Empirical Evaluation of Agile Processes, XP/Agile Universe*, 2002.
- [29] T. Abdel-Hamid, "The Dynamics of Software Projects Staffing: A System Dynamics Based Simulation Approach," *IEEE Trans. Software Eng.*, vol. 15, no. 2, pp. 109-119, Feb. 1989.
- [30] S. Kuppaswami, K. Vivekanandan, and P. Rodrigues, "A System Dynamics Simulation Model to Find the Effects of XP on Cost of Change Curve," *Proc. Fourth Int'l Conf. Extreme Programming and Agile Processes in Software Eng.*, May 2003.

- [31] V. Lepar and P.P. Shenoy, "A Comparison of Lauritzen-Spiegelhalter, Hugin, and Shenoy-Shafer Architectures for Computing Marginals of Probability Distributions," *Proc. 14th Conf. Uncertainty in Artificial Intelligence*, 1998.
- [32] *Agile Manifesto*, <http://www.agilemanifesto.org/>, accessed May 2007.
- [33] A. Cau, G. Concas, M. Melis, and I. Turnu, "Evaluate XP Effectiveness Using Simulation Modeling," *Proc. Sixth Int'l Conf. Extreme Programming and Agile Processes in Software Eng.*, June 2005.
- [34] B. George and L. Williams, "An Initial Investigation of Test-Driven Development in Industry," *Proc. ACM Symp. Applied Computing*, pp. 1135-1139, 2003.
- [35] H. Wang, F. Peng, C. Zhang, and A. Pietschker, "Software Project Level Estimation Model Framework Based on Bayesian Belief Networks," *Proc. Sixth Int'l Conf. Quality Software*, 2006.
- [36] L. Stamelos, P. Angelis, and E. Dimou, "On the Use of Bayesian Belief Networks for the Prediction of Software Productivity," *Information and Software Technology*, vol. 45, no. 1, pp. 51-60, 2003.
- [37] A. Albrecht and J. Gaffney, "Software Function, Source Lines of Code, and Development Effort Prediction: A Software Science Validation," *IEEE Trans. Software Eng.*, vol. 9, no. 6, pp. 639-648, 1983.
- [38] M. Cohn, *Agile Estimating and Planning*. Prentice Hall, 2005.
- [39] M. Jørgensen and M.J. Shepperd, "A Systematic Review of Software Development Cost Estimation Studies," *IEEE Trans. Software Eng.*, vol. 33, no. 1, pp. 33-53, Jan. 2007.
- [40] K. Moløkken and M. Jørgensen, "A Review of Surveys on Software Effort Estimation," *Proc. Int'l Symp. Empirical Software Eng.*, pp. 223-231, 2003.
- [41] M. Jørgensen and D.I.K. Sjøberg, "An Effort Prediction Interval Approach Based on the Empirical Distribution of Previous Estimation Accuracy," *Information and Software Technology*, vol. 45, no. 3, pp. 123-136, 2003.



Peter Hearty received the BSc degree in mathematics and physics from the University of Stirling in 1982. He is a research assistant and a PhD student at Queen Mary, University of London. He was a programmer, analyst, and designer for various commercial organizations before founding his own database company in 1997.



Norman Fenton is a professor of computer science at Queen Mary, University of London, where he is the director of the Risk Assessment and Decision Analysis Research (RADAR) Group. He is also the chief executive officer of Agena, a company that specializes in risk management for critical systems. He is an affiliated professor at the University of Haifa, Israel. He has held previous academic posts at City University (professor in the Centre for Software Reliability), South Bank (director of the Centre for Systems and Software Engineering), and Oxford University and University College Dublin (both as a research fellow). He was a visiting researcher at GMD, Bonn, Germany. He has published extensively in various areas of mathematics, software engineering, and risk analysis. His recent work has focused on causal models (Bayesian Nets) for risk assessment in a wide range of application domains.



David Marquez received the PhD degree in mathematics from the University of Marne la Vallée, France. He is a research assistant in the Risk Assessment and Decision Analysis (RADAR) Group in the Department of Computer Science at Queen Mary, University of London. Before joining academia, he was a senior researcher in the oil industry, developing mathematical and statistical models for reservoir characterization problems. His research interests include Bayesian statistical modeling, Bayesian networks, and statistical pattern recognition.



Martin Neil received the BSc degree in mathematics and the PhD degree in statistics and software metrics. He is a professor of computer science and statistics in the Department of Computer Science at Queen Mary, University of London, where he teaches decision and risk analysis and software engineering. He is also a joint founder and chief technology officer of Agena, which develops and distributes AgenaRisk, a software product for modeling risk and uncertainty. His interests include Bayesian modeling and/or risk quantification in diverse areas: operational risk in finance, systems and design reliability (including software), project risk, decision support, simulation, AI and personalization, and statistical learning. He is a chartered engineer.

► For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.